

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Boost Your AI Models: A Hands-On Guide to RAG and MCP

13 min read · Aug 6, 2025



Amit Verma

Follow



Listen



Share



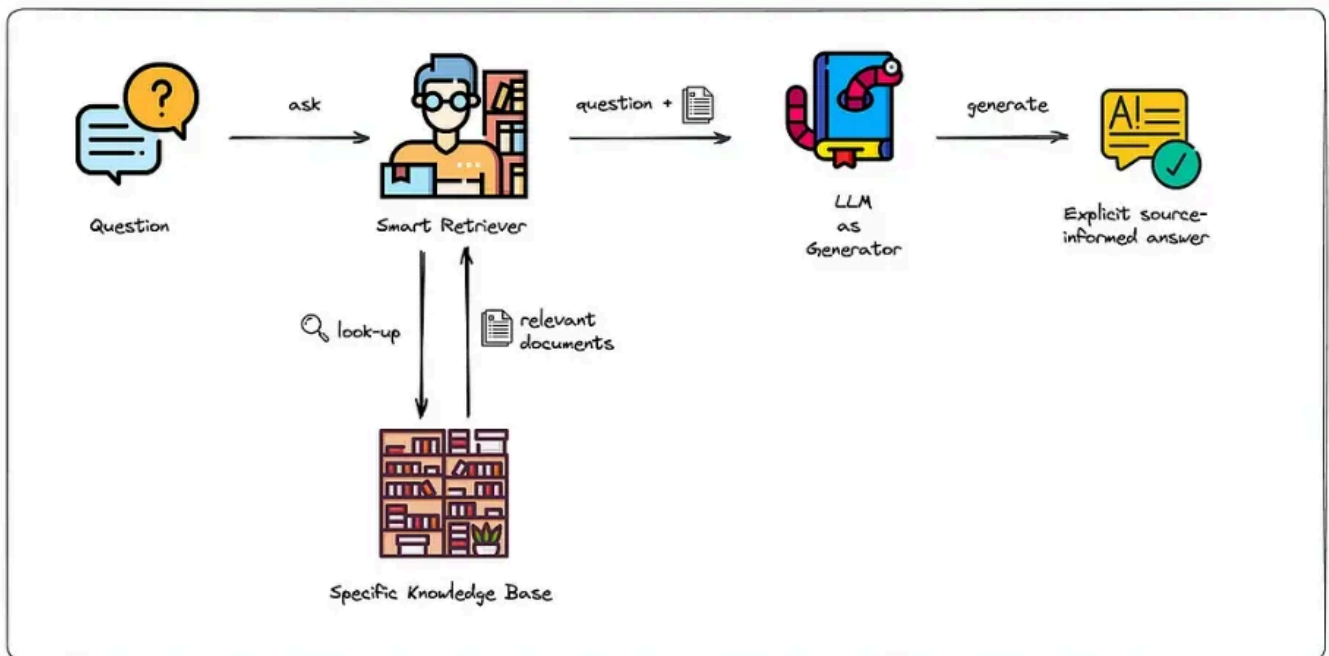
More

Retrieval-Augmented Generation and Memory-Conditioned Prompting.

What is RAG (Retrieval-Augmented Generation)?

RAG is an advanced architecture for large language models that combines the strengths of traditional language generation with external knowledge retrieval.

Instead of relying solely on LLMs(pre-trained knowledge), **RAG** pulls real-time data from external sources, ensuring more accurate and up-to-date responses. This makes AI models more reliable, especially for applications that require more accurate and up-to-date responses.



[image-source](#)

Why was RAG Invented?

Large Language Models (LLMs) like GPT-4 are powerful but have **two major limitations**:

1. **Static knowledge** — They are trained on a fixed dataset (knowledge cutoff), so they don't know recent events or private data.
2. **Hallucination** — They may confidently generate **factually incorrect or fabricated answers**.

Imagine that within your company you access an LLM like ChatGPT or Gemini and ask through the chat: “What was the company’s revenue result in August 2024?”

He probably won't know how to answer or will give a very imprecise answer, because either he doesn't have access to this information or it is public, but it is very recent information, this is where the RAG comes into play.

RAG was invented to solve:

- Lack of **domain-specific, real-time, or proprietary** knowledge in LLMs.
- Risk of **hallucination** by grounding the output in **retrieved documents**.
- Making LLMs **more trustworthy, transparent, and update-friendly**.

Real-World Analogy: Copilot + RFP Upload = RAG

When you upload an RFP and then ask questions:

- You're giving **unstructured data** (PDF/Doc/Markdown) to the system (example-RFP = knowledge base).
- It **retrieves relevant chunks** (e.g., “challenges”, “pain points”) based on your query.
- It **augments** your query with that retrieved knowledge.
- Then, the **LLM generates** a response using both your question and the retrieved knowledge.

This is **exactly what RAG does**:

RAG = Retrieval (fetch relevant info) + Generation (LLM uses that info to answer)

Retrieval-augmented generation (RAG) is a technique for enhancing the accuracy and reliability of generative AI models with facts fetched from external sources.

How RAG Works?

RAG combines **two systems**:

1. Retriever

- Fetches top-k relevant documents from an external **vector database** or external data source (e.g., PDF, database, knowledge base, etc.) based on the input query.
- Uses **embeddings** (via models like `sentence-transformers`) and semantic and similarity search (e.g., via **FAISS**, **Chroma**, **Weaviate**).

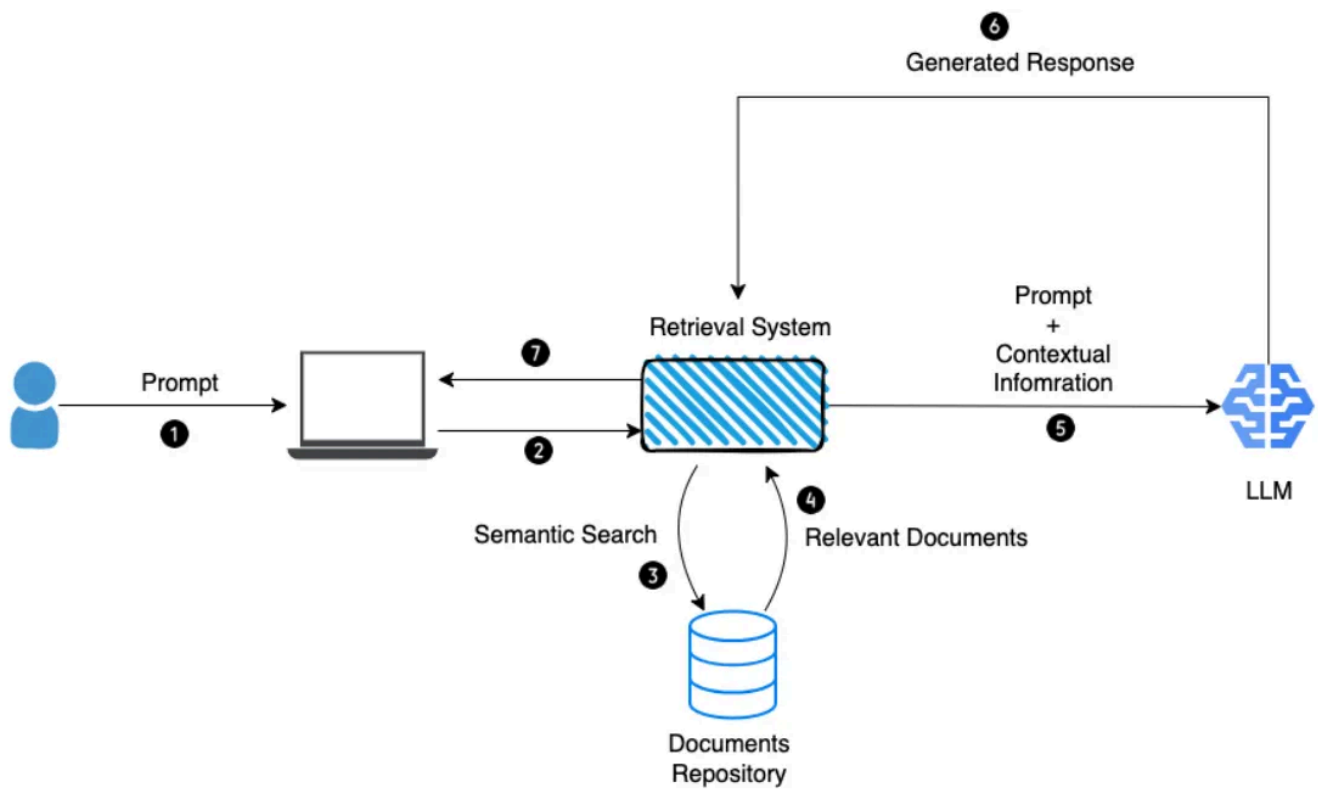
2. Augmentation (Fusion Phase)

- The retrieved information is then added to the original query, effectively “augmenting” the prompt given to the LLM.

3. Generator

- A language model (e.g., GPT-4, LLaMA, or FLAN-T5) receives the augmented prompt and generates a **final answer** grounded in the retrieved data.

User Query → Embedding → Similar Docs Retrieved →
Passed to LLM → Grounded Answer Generated



Retrieval-Augmented Generation

Click [here](#) to explore different types of RAG.

Use Case	Description	RAG Impact
 Healthcare Q&A	Doctors query medical literature for latest treatments.	Access to latest research, reduced hallucination.
 Legal Assistants	Lawyers search judgments, case law, or contracts.	Context-aware summarization, accurate citations.
 Enterprise Knowledge Bots	Employees ask HR or IT questions.	Accesses internal policies and SOPs.
 E-Commerce Chatbot	Users ask for products, warranty, offers.	Combines real-time inventory with LLMs.
 Educational Tutor	Students ask questions from textbooks or notes.	Personalized learning from course-specific data.

Implementation Steps

1. Document Preparation

- Collect data (PDFs, Excel, websites, etc.)
- Split into **chunks** (e.g., 500 words)
- Clean and preprocess

2. Embedding

- Use an embedding model to convert each chunk into a **vector**

3. Vector DB Setup

- Store these vectors using **FAISS, Chroma**, etc.
- Support fast **semantic similarity search**

4. Query Pipeline

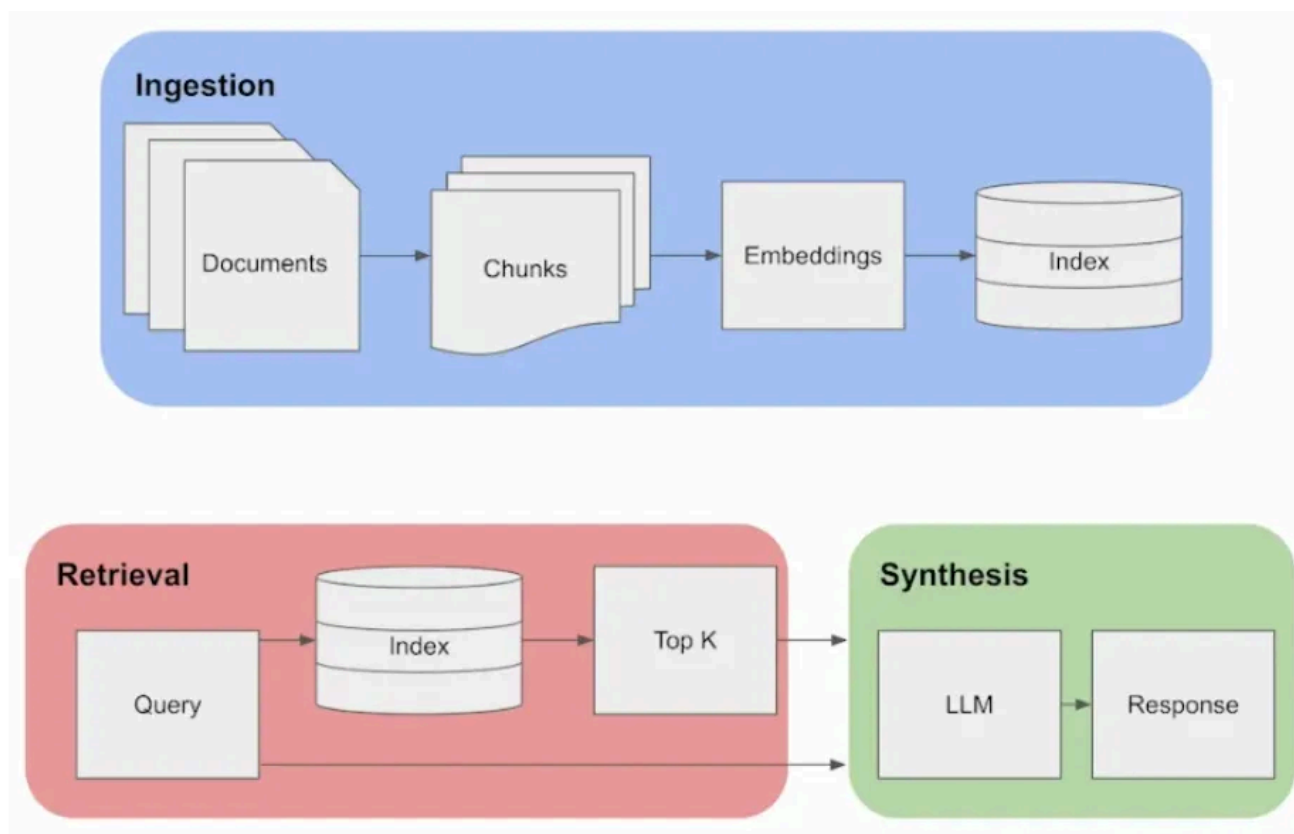
- User submits a query → convert to embedding → retrieve top-k chunks

5. LLM Generation

- Retrieved chunks + query → sent to LLM for response
- Response is **grounded**, accurate, and explainable

6. Optional UI

- Add a simple frontend (e.g., Gradio) for interaction



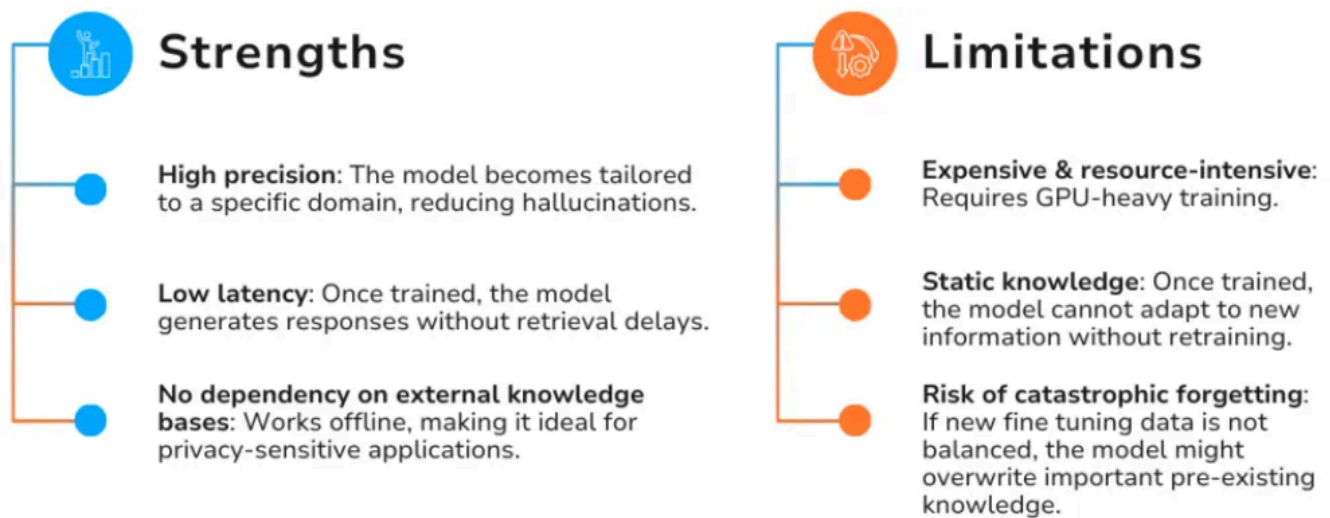
Basic RAG Pipeline

Real-World Example:

RAG Use Case: A legal assistant chatbot retrieves relevant case law or statutes to answer user queries, always referencing the latest legal content.

Simple Analogy- Think of RAG as:

Instead of making the LLM guess or hallucinate, RAG equips it with **factual references** before it answers.



RAG vs FINE-TUNING

Full Code Example: Document Analyzer with RAG

Here's a complete and generic “**Document Q&A App**” that supports PDF, DOCX, and Excel files, using **RAG (Retrieval-Augmented Generation)** and **Gemini API**, deployable in **Hugging Face Spaces** with **Gradio**.

Features:

- Upload PDF , DOCX , TXT , or Excel (.xlsx) file
- Ask questions based on uploaded content
- Uses **Gemini Pro** as LLM
- Works fully in Hugging Face Spaces (Gradio UI)

```
your_project_folder/  
├── app.py  
├── requirements.txt  
└── .gitattributes (optional)
```

📄 requirements.txt

```
langchain
langchain-community
langchain-google-genai
google-generativeai
pypdf
pandas
python-docx
openpyxl
chromadb
tiktoken
faiss-cpu
python-pptx
```

Get Gemini API Key

Go to <https://makersuite.google.com/app/apikey> and generate your API key.

Component	Tool/Option
Chunking & Parsing	LangChain , PyMuPDF , or pdfplumber
Embeddings	OpenAI , HuggingFace , Cohere , etc.
Vector DB	FAISS , Chroma , Pinecone , etc.
LLM	GPT-4 , Claude , LLama , etc.
Framework (optional)	LangChain , LlamaIndex , etc.
UI	Gradio , Streamlit , FastAPI , etc.



app.py – Full Code

```
import os
import gradio as gr
import google.generativeai as genai

from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.docstore.document import Document
from langchain_community.vectorstores import FAISS
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain.chains import RetrievalQA
from langchain_google_genai import GoogleGenerativeAI
```

```

from docx import Document as DocxDocument
import openpyxl
from pypdf import PdfReader
from pptx import Presentation
import tempfile

# 🗝️ Set your Gemini API key (no hardcoding, using Hugging Face secrets)
os.environ["GOOGLE_API_KEY"] = os.environ.get("GEMINI_API_KEY", "")
genai_api_key = os.environ["GOOGLE_API_KEY"]

# 📄 Load and process documents
def load_and_split_document(file):
    file_path = file.name
    ext = os.path.splitext(file_path)[1].lower()

    text = ""
    if ext == ".pdf":
        reader = PdfReader(file_path)
        text = "\n".join([page.extract_text() or "" for page in reader.pages])

    elif ext == ".docx":
        doc = DocxDocument(file_path)
        text = "\n".join([para.text for para in doc.paragraphs])

    elif ext == ".xlsx":
        # ✅ IMPROVED EXCEL PROCESSING
        wb = openpyxl.load_workbook(file_path)
        text = f"Excel File: {os.path.basename(file_path)}\n"
        text += f"Total Sheets: {len(wb.worksheets)}\n\n"

        for sheet_idx, sheet in enumerate(wb.worksheets, 1):
            text += f"=== SHEET {sheet_idx}: {sheet.title} ===\n"

            # Get all data from sheet
            data = []
            for row in sheet.iter_rows(values_only=True):
                if any(cell is not None and str(cell).strip() for cell in row):
                    data.append([str(cell) if cell is not None else "" for cell

            if data:
                # First row as headers
                if len(data) > 0:
                    headers = data[0]
                    text += f"Headers: {' | '.join(headers)}\n"

                # Data rows
                for row_idx, row in enumerate(data[1:], 1):
                    text += f"Row {row_idx}: {' | '.join(row)}\n"

                # Add some context every few rows
                if row_idx % 10 == 0:

```



```

        text += f"--- {row_idx} rows processed in sheet '{sheet.title}'\n"

        text += f"Total rows in '{sheet.title}': {len(data)-1}\n"
    else:
        text += f"No data found in sheet '{sheet.title}'\n"

    text += "\n" + "="*50 + "\n\n"

elif ext == ".pptx":
    prs = Presentation(file_path)
    for slide in prs.slides:
        for shape in slide.shapes:
            if hasattr(shape, "text"):
                text += shape.text + "\n"

elif ext == ".txt":
    with open(file_path, "r", encoding="utf-8") as f:
        text = f.read()

else:
    raise ValueError("Unsupported file type.")

# Split text into chunks
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
docs = text_splitter.create_documents([text])
return docs, text

# 🌟 Set up LLM and QA chain
def create_qa_chain(docs):
    embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001", google_api_key=google_api_key)
    vectorstore = FAISS.from_documents(docs, embeddings)
    retriever = vectorstore.as_retriever()

    llm = GoogleGenerativeAI(model="gemini-1.5-flash", google_api_key=google_api_key)
    qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
    return qa_chain, llm

# 🔍 Main QA logic
def process_file(file, question):
    try:
        # Initialize LLM
        llm = GoogleGenerativeAI(model="gemini-1.5-flash", google_api_key=google_api_key)

        # Case 1: No file uploaded → ask Gemini directly
        if file is None:
            try:
                response = llm.invoke(question)
                return f"(LLM Answer - No File)\n{response}"
            except Exception as e:
                if "quota" in str(e).lower() or "exceeded" in str(e).lower():
                    return "🚫 Token limit exhausted. Please try again later."
                return f"❌ Error: {str(e)}"

```

```

# Case 2: File uploaded → Enhanced RAG with smart fallback
docs, text = load_and_split_document(file)

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/embedding-001",
    google_api_key=genai_api_key,
)
vectorstore = FAISS.from_documents(docs, embeddings)
retriever = vectorstore.as_retriever()

qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
rag_answer = qa_chain.run(question)

# Enhanced fallback logic
fallback_triggers = [
    "i don't know", "i do not know", "not found", "cannot find",
    "no information", "not mentioned", "not available", "unclear",
    "insufficient", "don't have information", "do not have information",
    "does not contain", "is not mentioned", "not provided"
]

# Check if the answer contains uncertainty phrases
contains_uncertainty = any(trigger in rag_answer.lower() for trigger in

# Check if question seems completely irrelevant to document
irrelevant_phrases = [
    "does not contain", "is not mentioned", "not provided in",
    "you will need to search", "search online", "use a search engine",
    "check online", "look up", "not related to"
]

is_completely_irrelevant = any(phrase in rag_answer.lower() for phrase in

if contains_uncertainty or is_completely_irrelevant:
    try:
        # For completely irrelevant questions, just ask LLM directly
        if is_completely_irrelevant:
            response = llm.invoke(question)
            return f"🤖 **General Answer:**\n{response}"
        else:
            # For partially related questions, use hybrid approach
            hybrid_prompt = f"""
            I have uploaded a document that contains: {text[:500]}...

            My question: {question}

            Please provide a complete answer. Use information from the document
            and supplement with your general knowledge when the question is not
            fully covered by the document.

            """
            response = llm.invoke(hybrid_prompt)
            return f"📖 **Complete Answer (Document + General Knowledge):**\n{response}"
    except Exception as e:
        print(f"Error in fallback logic: {e}")
        return "I'm sorry, I couldn't provide a complete answer."

```

```

        except Exception as e:
            if "quota" in str(e).lower() or "exceeded" in str(e).lower():
                return "🚫 Token limit exhausted. Please try again later."
            return f"❌ Error: {str(e)}"

    return f"📄 **Answer from Document:**\n{rag_answer}"

except Exception as e:
    # Global error handler for any unexpected issues
    if "quota" in str(e).lower() or "exceeded" in str(e).lower():
        return "🚫 Token limit exhausted. Please try again later."
    return f"❌ Unexpected Error: {str(e)}"

# 🧠 Gradio UI
with gr.Blocks() as demo:
    gr.Markdown("### 📄 Document Q&A with Gemini (PDF, DOCX, XLSX, PPTX, TXT Support)")
    with gr.Row():
        file_input = gr.File(label="Upload File", file_types=[".pdf", ".docx", ".xlsx", ".pptx", ".txt"])
        question_input = gr.Textbox(label="Ask a Question", placeholder="E.g. What is the capital of France?")
    submit_btn = gr.Button("Submit")
    output = gr.Textbox(label="Answer")

    submit_btn.click(fn=process_file, inputs=[file_input, question_input], outputs=[output])

if __name__ == "__main__":
    demo.launch()

```

Step	Description
1. Load Docs	Use <code>PyPDFLoader</code> , etc.
2. Chunking	Split large docs into searchable pieces
3. Embedding	Convert text chunks into vectors
4. Vector DB	Store and retrieve via similarity
5. LLM	Combine query + chunk = final answer
6. UI	Gradio/Streamlit frontend for client demo

Deployment on Hugging Face Spaces

1. Go to Hugging Face Spaces
2. Create a new Space → choose **Gradio + Python**
3. Upload `app.py` and `requirements.txt`
4. Set your **Gemini API Key** as secret:
 - Go to **Settings** → **Secrets** → Add:

```
GEMINI_API_KEY = your_gemini_api_key
```

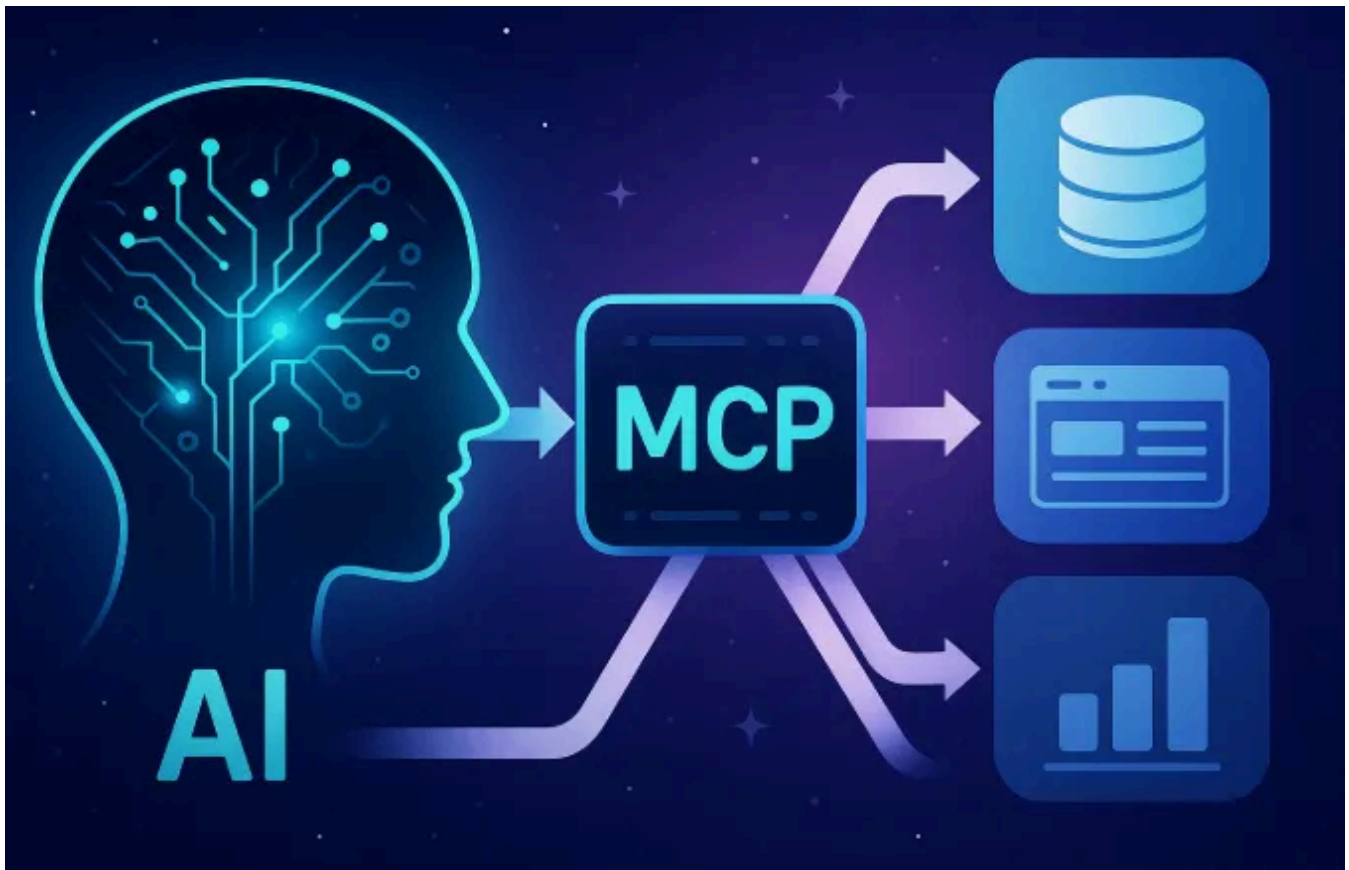
5. Deploy !

```
[ User Query ]
    |
    v
[ Embed Query → Vector ]
    |
    v
[ Search relevant chunks from Vector DB ]
    |
    v
[ Combine Query + Retrieved Context ]
    |
    v
[ Send to LLM → Get Answer ]
```

Conclusion for RAG

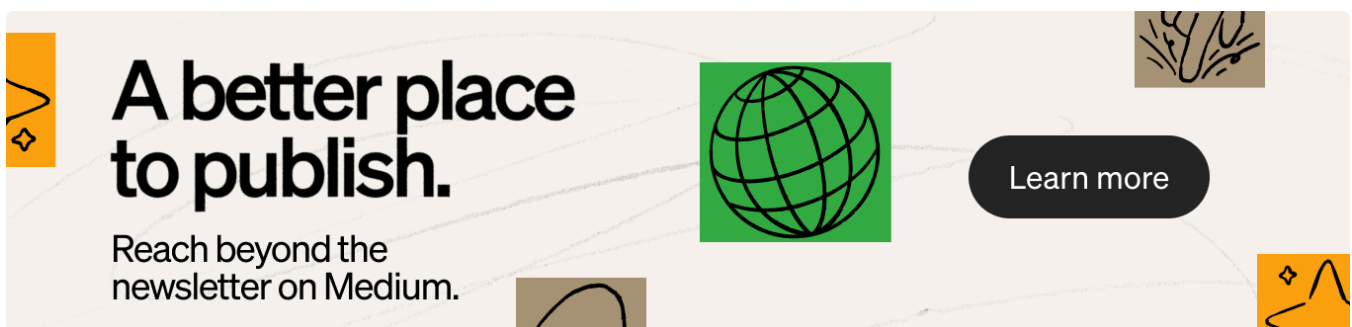
- RAG solves key LLM limitations: hallucination, outdated knowledge, and lack of domain-specific context.
- It combines **retrieval (search)** and **generation (LLMs)** to produce grounded, accurate, and explainable outputs.
- RAG enables **real-time** and **private knowledge integration** in AI systems.
- It is highly impactful in sectors like **healthcare**, **legal**, **enterprise support**, **education**, and **e-commerce**.

- Tools like LangChain, FAISS, Chroma, and LlamaIndex make RAG implementations accessible and practical.



What is MCP (Model Chaining Pattern)?

MCP stands for Model Chaining Pattern. It's a design pattern used in LLM (Large Language Model) applications where **multiple models (or prompts)** are connected in a **pipeline**. Each model (or prompt step) performs a **specific sub-task**, and the **output** of one becomes the **input** of the next.



MCP (Model Context Protocol) is like having a **universal translator** between AI models and external systems.

*Think of it as **HTTP for AI** — just as HTTP allows browsers and web servers to communicate, MCP allows AI models or agents to interact intelligently.*

MCP is an **open protocol** that standardizes how AI assistants connect to external data sources and tools. Think of it as:

- **USB for AI:** Just like USB standardized how devices connect to computers
- **HTTP for AI Tools:** A common language all systems can speak
- **The Universal Remote:** One interface to control all your smart home devices

Why was MCP Invented?

Problems with Traditional AI Usage:

- Most LLMs and models are **single-task and stateless** — they handle one query at a time.
- Complex tasks often need **multiple models or steps** (e.g., search → summarize → answer → visualize).
- Coordination between models was **manual, ad-hoc, and non-scalable**.

MCP was invented to:

- Enable **structured, scalable communication** between AI models.
- Create **multi-model pipelines** where each model/agent performs a task and passes control/data forward.
- Allow AI systems to **remember, reason, and coordinate** more like human teams.

Problem	MCP Solution
✗ Isolated AI models	✓ Seamless multi-model collaboration
✗ Stateless processing	✓ Context sharing and memory chaining
✗ Manual pipeline creation	✓ Standardized orchestration
✗ Lack of reasoning across steps	✓ Task coordination among intelligent agents
✗ Repetitive outputs in complex workflows	✓ Step-wise structured generation

Problems Solved by MCP

Is MCP about memory?

Yes, **partially**. But memory is **just one pillar** of MCP. MCP is **not only** about storing memory (like how RAG does).

*It is more about **standardizing the communication, coordination, and chaining** between multiple models and components using **shared context**.*

LLMs are powerful, but trying to solve **complex tasks in one shot (monolithic prompt)** often leads to:

- Confused or vague outputs
- High token usage
- Lack of control over intermediate steps
- Difficulty in debugging or improving accuracy

Just like we break down complex software problems into **modules or microservices**, we can break LLM workflows into **small, composable, manageable units** — this is where **MCP** helps.

Problem	How MCP Helps
Prompt complexity	Breaks one big task into smaller logical subtasks
Low reliability	Modular tasks are easier to control and test
Token wastage	Efficient by isolating subtasks with fewer tokens
No reusability	Each sub-model/task can be reused in other pipelines
Debugging difficulty	Easy to identify which step failed or returned poor results

How MCP Works? (Architecture)

1. **Model Chain** — A pipeline of models (or agents), each performing a specific role.
2. **MCP Server** — The orchestrator that manages communication, memory, and execution.
3. **MCP Message** — Structured message protocol that includes query, context, memory, and task instruction.

User Query → Model A (Research Agent) →
Model B (Summarization Agent) →

Components:

- **Input:** User prompt or task
- **Chain of Models:** LLMs or agents each with defined purpose
- **Shared Memory:** Persistent context/state passed across models
- **MCP Server:** The controller that tracks model outputs, inputs, metadata

Sequential MCP	Models are chained step-by-step ($A \rightarrow B \rightarrow C$)
Parallel MCP	Multiple models work concurrently ($A, B \rightarrow \text{Merge} \rightarrow C$)
Conditional MCP	Branching logic based on output ($A \rightarrow B \text{ or } C$)
Interactive MCP	Human-in-the-loop or feedback-driven chaining

Why was MCP needed? What problems does it solve?

The Problem- In traditional LLM apps:

- LLMs are **stateless** — they forget past context unless it is manually passed.
- **Multiple models** can't easily communicate or share memory.
- **Long chains** of models (like Agent → Planner → Executor → Analyzer) break if the context isn't shared correctly.
- Adding memory manually is **complex** and brittle.

The Solution: MCP

MCP offers a **structured** way to:

- Pass information (memory/context) between models.
- Maintain a **consistent context or knowledge graph**.
- Enable **intelligent orchestration** between agents and models.

Real-World Example: Product Launch AI

Imagine you're launching a **new product**. You assign the task to a **smart assistant (LLM)**.

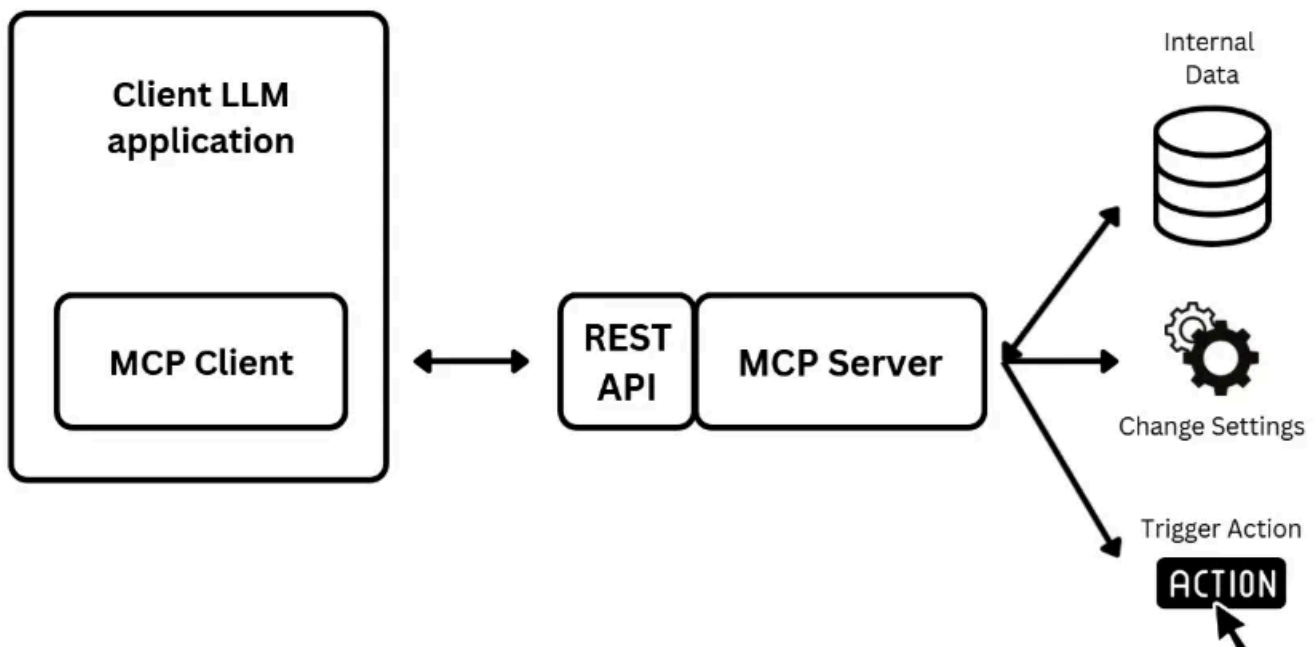
You say: *"Hey AI, launch a new AI-powered smartwatch."*

Without MCP, the LLM tries to do everything in one shot — market research, feature planning, pricing, etc. The result is often generic or shallow.

Now let's break it with **MCP**:

1. **Agent 1 — Market Research Agent:** "Analyze trends in wearable tech."
2. **Agent 2 — Product Planner:** "Based on trends, suggest 3 smartwatch features."
3. **Agent 3 — Marketing Writer:** "Write a blog post about this smartwatch."
4. **Agent 4 — Pricing Analyst:** "Suggest pricing based on features and competitors."

Each step becomes **easier, clearer, and smarter** because the LLM is focused on one task at a time. That's **Model Chaining Pattern**.



[image-source](#)

What is an MCP Server?

The **MCP Server** is the central orchestrator that:

- Routes requests between models/agents
- Stores and shares memory/context
- Logs state transitions and decisions
- Handles model versioning, API tokens, and fallback logic

Types of MCP Servers:

Type	Function
Local MCP Server	Self-hosted for testing or internal workflows
Cloud MCP Server	Scalable and remote (e.g., using FastAPI or Node)
Agent Frameworks with MCP Layer	LangGraph (LangChain), CrewAI, AutoGen, Autogen Studio

How to Implement and Use MCP?

Tools & Stack

- **Language Models:** GPT-4, Claude, LLaMA, FLAN-T5
- **Orchestration Frameworks:** LangGraph, AutoGen, CrewAI, OpenAgents
- **Memory & Context:** Redis, LangChain Memory, JSON store
- **Communication:** REST, WebSockets, or internal APIs

Implementation Steps

1. **Define the Task** (e.g., “Launch a new product”)
2. **Identify Agents or Models** (Research Agent, Marketing Agent, Design Agent)
3. **Design Chain** — Define the flow and communication structure
4. **Build MCP Server** — Use FastAPI or LangGraph to coordinate agents
5. **Enable Memory** — Store shared context, history, and outputs
6. **Run Chain** — Input user query → step-by-step agent execution
7. **Add Feedback Loops** (optional) — Allow agents to validate each other or retry

Real-World Examples of MCP

 Product Launch AI	Research Agent → Marketing Agent → Design Agent	End-to-end product planning
 Invoice Analyzer	File Uploader → OCR Model → Classifier → Extractor	Automates document processing
 HR Interview Bot	Resume Analyzer → Score Generator → Email Generator	Speeds up hiring
 Client Proposal Builder	Requirement → Generator → PPT Creator	AI-generated business proposals

POC: Startup Idea Evaluator with MCP

User submits a startup idea. The app will:

1. Extract main concept
2. Do market viability check
3. Suggest 3 competitors
4. Give final verdict (Good, Needs Refinement, Not Viable)

Code (Hugging Face + Gradio + Gemini API)

```
import os
import gradio as gr
from langchain.prompts import PromptTemplate
from langchain_google_genai import GoogleGenerativeAI

os.environ["GOOGLE_API_KEY"] = os.environ["GEMINI_API_KEY"]

llm = GoogleGenerativeAI(model="gemini-pro")

# Step 1: Extract Concept
def extract_concept(idea):
    prompt = PromptTemplate.from_template(
        "Extract the main concept from this startup idea: {idea}"
    )
    return llm.invoke(prompt.format(idea=idea))

# Step 2: Market Viability
def market_analysis(concept):
    prompt = PromptTemplate.from_template(
        "Analyze the market viability of this concept: {concept}"
    )
    return llm.invoke(prompt.format(concept=concept))

# Step 3: Competitor Suggestion
```

```

def suggest_competitors(concept):
    prompt = PromptTemplate.from_template(
        "List 3 competitors for this startup concept: {concept}"
    )
    return llm.invoke(prompt.format(concept=concept))

# Step 4: Final Verdict
def verdict(concept, market, competitors):
    prompt = PromptTemplate.from_template(
        "Based on this concept: {concept}, market analysis: {market}, and competitors: {competitors}"
    )
    return llm.invoke(prompt.format(concept=concept, market=market, competitors=competitors))

# MCP Workflow
def startup_evaluator(idea):
    try:
        concept = extract_concept(idea)
        market = market_analysis(concept)
        competitors = suggest_competitors(concept)
        final_opinion = verdict(concept, market, competitors)

        return f"""🧠 **Concept:** {concept}
        📊 **Market Analysis:** {market}
        🏢 **Competitors:** {competitors}
        ✅ **Verdict:** {final_opinion}"""
    except Exception as e:
        return f"❌ Error: {str(e)}"

# Gradio UI
gr.Interface(
    fn=startup_evaluator,
    inputs=gr.Textbox(label="Enter your startup idea"),
    outputs=gr.Markdown(),
    title="🚀 Startup Idea Evaluator (MCP powered)",
    description="Evaluates your startup idea using multi-step LLM pipeline based on MCP"
).launch()

```

Deploy to Hugging Face

```

gradio
langchain
langchain-google-genai

```

Conclusion on MCP

- **What:** MCP is a **protocol to connect multiple AI models or agents** in a structured, intelligent workflow.
- **Why:** Invented to **enable collaboration, context sharing, and reasoning** across multiple AI components.
- **Benefit:** Helps **build complex multi-step AI systems** that can reason and remember across steps.
- **Benefit:** Allows **reuse and combination** of multiple specialized models or tools.
- **Use Case:** Powers agentic workflows like **product launch, document processing, and R&D**.
- **How:** Implemented using tools like **LangGraph, AutoGen, FastAPI**, with memory and logic control via [MCP Server](#).
- **Impact:** MCP makes AI systems **modular, intelligent, explainable, and collaborative** — enabling the future of **Agentic AI**.

Further Reads-

What is Retrieval-Augmented Generation (RAG)?

Read about what is RAG and how to set up a Basic RAG Pipeline using Langchain, OpenAI LLM and ChromaDB. Sample code...

drjulija.github.io

RAG em LLMs: Entendendo o Retrieval-Augmented Generation - IA Com Café

Descubra como a arquitetura RAG (Retrieval-Augmented Generation) combina a busca semântica com grandes modelos de...

iacomcafe.com.br

Vectorize

Agentic AI Data Platform

vectorize.io

What is Retrieval-Augmented Generation(RAG) in LLM and How it works?

medium.com

RAG vs Agentic RAG vs MCP: A 2025 Comparison Guide for Business Leaders

RAG vs Agentic RAG vs MCP-explained for business leaders. Learn which AI approach fits your 2025 strategy in this quick...

www.bitcot.com

- + AI
- + Mcp Server
- + Agentic Ai
- + Machine Learning
- + LLM



Follow

Written by Amit Verma

632 followers · 23 following

Senior Technology Architect

